



UNIVERSITÀ DEGLI STUDI DI CATANIA

Dipartimento di Ingegneria Elettrica Elettronica e Informatica

Ingegneria informatica LM-32: Advanced Programming Languages

ByteGuard

Autori:

Riccardo Maria Villaggio & Giuseppe Ravesi

21 settembre 2026

Indice

1	Architettura del Sistema e Sviluppo dei Moduli	2
1.1	Architettura del Sistema e Suddivisione del Lavoro	2
1.2	Modulo Interfaccia e Orchestrazione (C# / WPF)	2
1.2.1	Inter-Process Communication (IPC) e Concorrenza	2
1.2.2	Modellazione dei Dati e Immutabilità (Record Types)	2
1.2.3	Gestione degli Eventi e Pattern Subscriber	3
1.3	Modulo di Cifratura e Autenticazione (C++)	3
1.3.1	Cifratura e Formato Proprietario (.lock)	3
1.3.2	Integrity Check (FNV-1a)	3
1.3.3	C++ Moderno e Gestione della Memoria (RAII)	3
1.4	Modulo di Monitoraggio e Watchdog (Go)	4
1.4.1	Rilevamento in Hot-Folder (Polling Temporizzato)	4
1.4.2	Concorrenza e Worker Pool (Produttore-Consumatore)	4
1.4.3	IPC e Graceful Shutdown	4
1.5	Motore di Analisi Forense (Python)	4
1.5.1	Analisi dell'Entropia di Shannon	4
1.5.2	Verifica della Coerenza dei Magic Bytes e Spoofing	5
1.5.3	Isolamento dell'I/O ed Error Handling	5

1 Architettura del Sistema e Sviluppo dei Moduli

1.1 Architettura del Sistema e Suddivisione del Lavoro

L'applicativo *ByteGuard* è concepito come un *tool* forense e di sicurezza a carattere multi-linguaggio, la cui architettura si articola in quattro moduli indipendenti, ciascuno implementato nel linguaggio più idiomatrico per il dominio di competenza. L'interoperabilità tra i moduli è garantita da un meccanismo di **Inter-Process Communication** (IPC) fondato sugli *Standard Stream* (I/O) e sulla serializzazione in **JSON puro**, evitando dipendenze da protocolli binari o framework di rete e massimizzando la portabilità del sistema. Lo sviluppo è stato ripartito equamente tra i due componenti del gruppo. **Giuseppe Ravesi** ha curato il modulo di *Orchestrazione e Interfaccia Utente* (C# / WPF) e il *Motore di Analisi Forense* (Python). **Riccardo Maria Villaggio** ha sviluppato il *Modulo di Cifratura e Autenticazione* (C++) e il *Watchdog di Monitoraggio* (Go). Le sezioni che seguono descrivono nel dettaglio le scelte architetturali e implementative di ciascun modulo. Il codice prodotto è disponibile al seguente link: <https://github.com/ByteGuard-APL/ByteGuard>

1.2 Modulo Interfaccia e Orchestrazione (C# / WPF)

Il modulo C# rappresenta il nucleo centrale del sistema, fungendo da orchestratore per l'intera architettura multi-linguaggio. Sviluppato avvalendosi del framework **Windows Presentation Foundation (WPF)** su ambiente .NET 10.0, il modulo si fa carico della gestione del ciclo di vita dei processi esterni (Go e C++), del data binding e dell'aggiornamento reattivo dell'interfaccia.

1.2.1 Inter-Process Communication (IPC) e Concorrenza

Per integrare gli eseguibili esterni senza ricorrere a complesse architetture di rete o socket, si è optato per una comunicazione IPC basata sugli *Standard Stream* (`stdout`). L'avvio del demone di monitoraggio in Go avviene in un task asincrono generato tramite la classe `ProcessStartInfo`. Particolare attenzione è stata posta alla gestione del *Current Working Directory* per garantire l'isolamento e la portabilità del deploy: l'eseguibile viene lanciato direttamente dalla `BaseDirectory` dell'`AppDomain` C#.

La lettura dell'output è implementata in streaming asincrono (`ReadLineAsync()`). Poiché il framework WPF impedisce l'aggiornamento dei componenti grafici da thread secondari (sicurezza del contesto UI), è stato impiegato il **Dispatcher**. Sfruttando delegati di tipo `Action` (mediante espressioni Lambda), il thread di background in ascolto delega al thread principale l'inserimento dei nuovi dati, garantendo una UI costantemente fluida.

1.2.2 Modellazione dei Dati e Immutabilità (Record Types)

I dati provenienti dal flusso IPC sono strutturati in formato JSON. Per la mappatura interna di tali strutture, si è rigettato l'uso di tuple o classi tradizionali in favore dell'adozione dei **Record** (introdotti in C# 9.0). La classe `AnalysisResult` è stata implementata come un *reference type record* definendo le proprietà in modalità `init-only`:

```
1 public record AnalysisResult
2 {
3     public string FilePath { get; init; }
4     public double ShannonEntropy { get; init; }
5     // ...
6 }
```

Questa scelta ingegneristica persegue un duplice obiettivo: da un lato garantisce la corretta serializzazione dinamica del JSON (delegando a `System.Text.Json` la mappatura automatica delle proprietà), dall'altro impone l'immutabilità post-inizializzazione. I dati forensi, una volta acquisiti, diventano di sola lettura, proteggendo il sistema da alterazioni accidentali. L'utilizzo del

costruito record garantisce altresì l'uguaglianza per valore (*value-based equality*), ottimizzando eventuali controlli di ridondanza.

1.2.3 Gestione degli Eventi e Pattern Subscriber

L'interazione con l'utente fa un uso estensivo dei delegati di sistema `EventHandler`. Il progetto adotta un pattern Publisher/Subscriber in cui i componenti grafici (aree di Drag & Drop, pulsanti di cifratura) fungono da publisher, e il codice *Code-Behind* vi si iscrive iniettando le logiche di risposta. Per la validazione dinamica dei dati a schermo, inoltre, è stata implementata l'interfaccia `IValueConverter`, al fine di formattare grandezze in byte o convertire esiti dell'analisi in indicatori visivi di allarme.

1.3 Modulo di Cifratura e Autenticazione (C++)

Il modulo C++ assolve alla funzione di cifratura e decifratura dei file, esponendo al contempo un meccanismo di verifica di integrità. È progettato per operare con complessità spaziale costante indipendentemente dalla dimensione dell'input, adottando un formato file proprietario che incapsula metadati e payload in un'unica unità atomica.

1.3.1 Cifratura e Formato Proprietario (.lock)

Il modulo implementa una cifratura simmetrica **XOR** eseguita *in-place*. Il risultato viene serializzato in un formato file proprietario con estensione `.lock`, la cui struttura è definita da un *header* binario custom scritto a basso livello tramite `reinterpret_cast`. L'header contiene i *Magic Bytes* identificativi (`BGUARD01`), la dimensione originale del file e il nome del file sorgente, consentendo la ricostruzione fedele del documento in fase di decifratura. Per garantire una complessità spaziale $O(1)$ in RAM — requisito critico nell'analisi di file di dimensioni arbitrariamente elevate — l'I/O su disco avviene tramite **elaborazione a chunk** di 4 KB. Il file non viene mai caricato interamente in memoria: ogni blocco è letto, cifrato e scritto sequenzialmente, rendendo il consumo di risorse indipendente dalla dimensione dell'input.

1.3.2 Integrity Check (FNV-1a)

Al fine di prevenire manipolazioni del contenuto cifrato o tentativi di decifratura con chiave errata, il modulo implementa manualmente l'algoritmo di hashing non crittografico **FNV-1a** nella variante a 64 bit. L'hash del *plaintext* originale viene calcolato durante la fase di cifratura e persistito all'interno dell'header `.lock`. In fase di decifratura, l'hash viene ricalcolato sul contenuto ricostruito e confrontato con il valore atteso: una discordanza segnala univocamente un'alterazione del payload o un errore nella chiave fornita, fungendo da *checksum* di autenticazione.

1.3.3 C++ Moderno e Gestione della Memoria (RAII)

Il modulo fa un uso estensivo delle funzionalità offerte dallo standard **C++20**. La gestione della memoria è interamente delegata al pattern **RAII** (*Resource Acquisition Is Initialization*): tutti i buffer sono allocati tramite `std::vector` e tutti gli stream di I/O sono gestiti mediante `std::ifstream`/`std::ofstream`. L'assenza totale degli operatori `new` e `delete` nel codice sorgente garantisce l'*Exception Safety* ed elimina alla radice ogni rischio di *memory leak*. Ulteriori accorgimenti includono l'adozione di `std::span` per ottenere *view* non-owning sicure sui buffer, l'impiego di `constexpr` in sostituzione delle macro del preprocessore, e una netta separazione architetturale tra *aggregati di dati* (`struct` POD) e *incapsulamento della logica* (`class`).

1.4 Modulo di Monitoraggio e Watchdog (Go)

Il modulo Go opera come *watchdog* di sistema, incaricato del rilevamento in tempo reale di variazioni nel file system e della conseguente orchestrazione delle analisi forensi. L'architettura sfrutta le primitive di concorrenza native del linguaggio per massimizzare il throughput su hardware multi-core.

1.4.1 Rilevamento in Hot-Folder (Polling Temporizzato)

Il monitoraggio della cartella sorvegliata (*Hot-Folder*) è realizzato mediante un **polling temporizzato**, implementato tramite `time.NewTicker`, al fine di evitare *leak* di memoria tipici delle interazioni prolungate con il *Garbage Collector*. Oltre alla semplice presenza di nuovi file, il modulo ispeziona il *timestamp* `ModTime` di ciascun elemento e adotta una logica a **differenza di insiemi** — realizzata tramite `map` Go utilizzate come *Set* — per classificare in tempo reale tre tipologie di eventi: file *nuovi*, file *modificati* (ad esempio a seguito di infezione) e file *cancellati*.

1.4.2 Concorrenza e Worker Pool (Produttore-Consumatore)

Trattandosi di invocazioni *CPU/I/O bound*, il modulo adotta il pattern architetturale **Worker Pool** (Produttore-Consumatore). Il ciclo di *polling* (produttore) invia i percorsi dei file rilevati su un *channel* bufferizzato, gestendo nativamente la *backpressure* in caso di saturazione della coda. Un pool di **goroutine**, dimensionato proporzionalmente al numero di core logici della macchina, consuma i messaggi dal canale e invoca i sottoprocessi Python di analisi. Questa architettura applica il principio idiomatrico del Go: “*share memory by communicating*”.

1.4.3 IPC e Graceful Shutdown

La comunicazione con la GUI C# avviene tramite emissione di payload JSON su `stdout`. Poiché l'accesso allo *standard output* è condiviso tra molteplici goroutine, ogni operazione di scrittura è protetta da un `sync.Mutex`, prevenendo l'*interleaving* dei byte e garantendo l'atomicità di ciascun messaggio. Lo spegnimento ordinato (*Graceful Shutdown*) è implementato intercettando i segnali del sistema operativo (`SIGINT`, `SIGTERM`). Alla ricezione del segnale, il modulo avvia la dismissione coordinata delle goroutine e dei canali tramite l'uso combinato di `select` e `sync.WaitGroup`, assicurando che ogni analisi in corso venga completata prima della terminazione del processo.

1.5 Motore di Analisi Forense (Python)

Il modulo Python agisce da analizzatore tattico *on-demand*. Attivato via CLI con il percorso del file da ispezionare, è progettato per compiere verifiche forensi mirate a identificare artefatti o anomalie comunemente associate ad attacchi malevoli.

1.5.1 Analisi dell'Entropia di Shannon

Uno degli algoritmi principali realizzati nel modulo è il calcolo dell'Entropia di Shannon, utilizzato per identificare file compressi subdolamente, frammenti crittografati o *packer* (tipici dei payload ransomware). Il calcolo valuta la distribuzione statistica e la casualità dei byte all'interno del file. Per bilanciare accuratezza forense e reattività d'uso, è stato implementato un meccanismo di tolleranza hardware: se il file in analisi risulta avere una dimensione superiore ai 100 MB, il modulo Python evita di saturare la memoria (OOM) commutando automaticamente su una strategia di **campionamento a blocchi** (*sampling*), calcolando un'entropia altamente rappresentativa senza bloccare l'I/O del disco.

1.5.2 Verifica della Coerenza dei Magic Bytes e Spoofing

Per individuare tentativi di camuffamento, il sistema estrae i primi 16 byte dell'intestazione del file (il *Magic Number*). Questo identificatore inalterabile viene confrontato con un dizionario interno mappato sull'estensione dichiarata dal file. Nel caso in cui il file dichiari di essere un `.pdf` ma il suo header corrisponda ad un binario eseguibile (`MZ`), l'applicativo marca il file come anomalo.

A rinforzo di questa verifica, è stato introdotto un controllo euristico sulle **doppie estensioni** (es. `documento.pdf.exe`). Il parser Python suddivide la *filename string* valutando le collisioni tra estensioni sicure (maschera) ed estensioni eseguibili reali, segnalando tempestivamente il tentativo tramite l'apposito flag `has_double_extension`.

1.5.3 Isolamento dell'I/O ed Error Handling

Il modulo Python è progettato per operare in un'architettura IPC, rendendo essenziale che il suo output sul canale `stdout` sia esclusivamente formattato in JSON puro. Per evitare che messaggi di debug, stack trace di eccezioni o warning di librerie terze corrompano il parser in ricezione (causando il collasso a catena del demone Go o della UI in `C#`), si è implementato un dirottamento globale dei log di sistema: eventuali warning diagnostici vengono incanalati in un file fisico separato (`byteguard_warnings.log`), assicurando un disaccoppiamento totale tra telemetria di debug e payload IPC di produzione.